

Travaux Pratiques : Administration PostgreSQL (r310tp1)

Partie 1 : Installation du service PostgreSQL

La première étape consiste à installer le système de gestion de bases de données relationnelles PostgreSQL ainsi que ses utilitaires.

```
root@rt-mv-xu26:~# apt install postgresql postgresql-contrib postgresql-client
root@rt-mv-xu26:~# systemctl status postgresql
```

Une fois le service démarré, nous accédons à l'invite de commande PostgreSQL pour modifier le mot de passe de l'administrateur et créer une nouvelle base de données.

```
root@rt-mv-xu26:~# sudo -u postgres psql postgres=# \password postgres Enter new
password for user "postgres": Azure1234 Enter it again: Azure1234 postgres=#
CREATE DATABASE rt3; CREATE DATABASE postgres=# \c rt3 You are now connected to
database "rt3" as user "postgres". rt3=#
```

Partie 2 : Création des utilisateurs et du schéma

Définition d'un schéma : Un schéma est un espace logique (similaire à un dossier) à l'intérieur de la base de données (ici `rt3`). Il permet de regrouper et d'isoler des tables.

Commandes utiles : `\du` permet de lister les rôles (utilisateurs), et `\dn` permet de lister les schémas existants.

```
rt3=# CREATE USER invite WITH PASSWORD 'Azerty123!'; CREATE ROLE rt3=# \du ...
(Liste des rôles, 'invite' apparaît désormais) ... rt3=# CREATE SCHEMA forum;
CREATE SCHEMA rt3=# \dn ... (Affiche 'forum' et 'public') ...
```

Partie 3 : Création des tables

Nous créons la table per sonne dans le schéma forum avec les contraintes suivantes :

- **SERIAL PRIMARY KEY** : auto-incrémentation pour générer un identifiant unique absolu pour chaque ligne.
- **VARCHAR(50)** : chaîne de caractères limitée à 50.
- **UNIQUE** : empêche d'insérer un pseudo qui existe déjà dans la base.
- **DATE** : stocke une date au format AAAA-MM-JJ.

```
rt3=# CREATE TABLE forum.personne ( id_personne SERIAL PRIMARY KEY, nom
VARCHAR(50), prenom VARCHAR(50), pseudo VARCHAR(50) UNIQUE, date_naissance DATE );
CREATE TABLE
```

Création de la table message avec une clé étrangère :

INT REFERENCES : Définit une clé étrangère. Cela vérifie que le numéro id_personne existe bel et bien dans la table per sonne. Si ce n'est pas le cas, le message est refusé par la base. **TEXT** permet de stocker des chaînes d'une longueur illimitée.

```
rt3=# CREATE TABLE forum.message ( id_message SERIAL PRIMARY KEY, id_personne INT
REFERENCES forum.personne(id_personne), contenu TEXT ); CREATE TABLE
```

Vérifications des tables créées :

```
rt3=# \dt forum.* rt3=# \d forum.personne rt3=# \d forum.message
```

Partie 4 : Insertion et vérification des données

```
rt3=# INSERT INTO forum.personne (nom, prenom, pseudo, date_naissance) VALUES ('Le
Meillat', 'Briac', 'networkbriac', '2025-04-01'), ('Turing', 'Alan', 'enigma',
'1972-06-03'); INSERT 0 2 rt3=# SELECT * FROM forum.personne;
```

Lors de l'insertion dans la table message, une erreur de frappe sur le nom de la colonne (id_pesonne) s'est révélée. Nous la corrigeons avec ALTER TABLE :

```
rt3=# ALTER TABLE forum.message RENAME COLUMN id_pesonne TO id_personne; ALTER
TABLE rt3=# INSERT INTO forum.message (id_personne, contenu) VALUES (1, 'dematt
world !'), (2, 'il n aura fallu que 2 mots pour casser enigma'); INSERT 0 2 rt3=#
SELECT * FROM forum.message;
```

Partie 5 : Gestion des privilèges et accès utilisateur

L'objectif est de donner l'accès à l'utilisateur invite sur le schéma forum et ses tables. L'attribut USAGE est la "clé de la porte", sans ça invite ne peut même pas voir que les tables existent.

```
rt3=# GRANT USAGE ON SCHEMA forum TO invite; GRANT rt3=# GRANT INSERT ON
forum.message TO invite; GRANT
```

Si le mot de passe n'avait pas été défini ou oublié, on peut le faire ainsi :

```
ALTER USER invite WITH PASSWORD 'Azerty123!';
```

Il est impératif d'autoriser également invite à utiliser la **séquence** permettant de générer automatiquement la clé primaire id_message, et de lui donner les droits de suppression et de sélection :

```
rt3=# GRANT USAGE ON SEQUENCE forum.message_id_message_seq TO invite; rt3=# GRANT
DELETE ON forum.message TO invite; rt3=# GRANT SELECT ON forum.message TO invite;
```

Tests des permissions en tant qu'invite

On bascule sur la session d'utilisateur invite (avec son propre mot de passe) :

```
root@rt-mv-xu26:~# psql -U invite -d rt3 -h 127.0.0.1 rt3=> INSERT INTO
forum.message (id_personne, contenu) VALUES (1, 'message test par invite'); INSERT
0 1 rt3=> DELETE FROM forum.message WHERE id_message = 3; DELETE 1 rt3=> SELECT *
FROM forum.personne; ERROR: permission denied for table personne
```

L'erreur sur la table personne est **attendue** car aucun droit SELECT n'a été accordé à invite sur cette table.

Vérification de la matrice des droits

En se reconnectant en tant que postgres (le superutilisateur), nous pouvons vérifier la matrice des permissions avec la commande \z.

```
rt3=# \c rt3 postgres rt3=# \z forum.* ... On observe que dans "Access
privileges", le champ pour "forum | personne" est vide pour 'invite'. Les
privilèges par défaut s'appliquent (aucun droit). ...
```

Partie 6 : Les sauvegardes (dumps) et restaurations

Nous allons extraire des sauvegardes du schéma via l'utilitaire pg_dump, en appelant cet outil avec les privilèges sudo appropriés.

Exportation complète : Sauvegarde de la structure ET des données du schéma.

```
root@rt-mv-xu26:~# sudo -u postgres pg_dump -d rt3 -n forum > forum_complet.sql
root@rt-mv-xu26:~# ls forum_complet.sql ...
```

Exportation Structure seule (-s) et Données seules (-a) :

```
root@rt-mv-xu26:~# sudo -u postgres pg_dump -d rt3 -n forum -s >
forum_structure.sql root@rt-mv-xu26:~# sudo -u postgres pg_dump -d rt3 -n forum -a
> forum_donnees.sql
```

Tests de restauration

Nous préparons des schémas d'accueil pour tester les restaurations.

```
root@rt-mv-xu26:~# sudo -u postgres psql -d rt3 -c "CREATE SCHEMA backup;" CREATE
SCHEMA root@rt-mv-xu26:~# sudo -u postgres psql -d rt3 -c "CREATE SCHEMA
fullrestore;" CREATE SCHEMA
```

Pour restaurer notre sauvegarde complète dans un schéma différent (fullrestore), nous remplaçons dynamiquement l'occurrence "forum" dans le script SQL à l'aide de l'outil texte sed, puis nous l'injectons dans psql :

```
root@rt-mv-xu26:~# sed 's/forum/fullrestore/g' forum_complet.sql | sudo -u
postgres psql -d rt3
```

Enfin, nous vérifions depuis psql l'état final des schémas :